Leitura — Fluxo de trabalho: Lean, Kanban, métricas e retrospectivas (Sprint 1)

Guia de apoio para as duas aulas da Sprint 1: Lean e Kanban (14/09) e métricas de fluxo, gestão visual e retrospectivas (21/09). Na Sprint 0, a pergunta era "como nos organizamos em ciclos" (Scrum e XP). Nesta, é "como o trabalho flui, e onde ele trava". A entrega da sprint cobra exatamente isso: quadro com limites de WIP em uso real, uma prática de XP com evidência, métricas e uma retrospectiva com ações.

Os slides das aulas são `slides/processos-slides-04.md` e `slides/processos-slides-05.md`. Os números dos exemplos foram calculados à mão e podem ser refeitos com uma planilha.

Como ler: os capítulos 2 a 6 são de 14/09; os capítulos 7 a 9, de 21/09. O capítulo 10 junta tudo no GitHub Projects.

Capítulos:

1. Onde a Sprint 1 se encaixa
2. Lean: valor e desperdício
3. Os sete desperdícios e os sete princípios no software
4. Kanban: gerir o fluxo
5. Visualizar e tornar as políticas explícitas
6. Limitar o WIP e a Lei de Little
7. Métricas de fluxo
8. Gestão visual e gargalos
9. Retrospectivas
10. No projeto: o GitHub Projects
11. Exercícios e dúvidas frequentes

---

1. Onde a Sprint 1 se encaixa

Na Sprint 0, a equipe escreveu um acordo de processo: cadência, cerimônias, Definição de Pronto, papéis, ferramentas e limites de WIP. Um acordo escrito ainda não é processo. A Sprint 1 é o primeiro confronto do acordo com o trabalho real:

- o quadro mostra onde o trabalho está e onde ele para;
- os limites de WIP obrigam a terminar antes de começar;
- as métricas dizem quanto tempo as coisas levam, com números em vez de impressões;
- a retrospectiva transforma o que foi observado em mudanças concretas no acordo.

Scrum e Kanban não competem aqui. O ritmo continua o do Scrum (sprints com início, fim, revisão e retrospectiva); o Kanban entra como a forma de gerir o fluxo dentro da sprint, sobre o mesmo quadro.

---

2. Lean: valor e desperdício

2.1 De onde vem

Lean nasce no Sistema Toyota de Produção, no Japão do pós-guerra, e ganha o nome em estudos do MIT nos anos 1990. De sistema de fábrica, virou uma forma de gerir qualquer trabalho em que valor flui até um cliente, inclusive software.

A ideia central é olhar o fluxo de valor: o caminho que um pedido percorre até virar algo útil nas mãos de quem pediu. Tudo o que não agrega valor nesse caminho é candidato a desperdício. Uma imagem usada nas aulas: numa corrida de revezamento, observe o bastão, não os corredores. Corredores ocupados não garantem que o bastão chega rápido.

2.2 Os dois pilares

- Respeito pelas pessoas: quem faz o trabalho participa das decisões sobre como fazê-lo; gestores ensinam em vez de mandar.
- Melhoria contínua (kaizen): pequenas mudanças, sempre, testadas e medidas.

Sem os pilares, as ferramentas do Lean viram ritual. Um quadro bonito numa equipe que não pode mudar o próprio processo não melhora nada.

2.3 Muda, mura e muri

O Lean fala de três fontes de desperdício:

Termo	O que é	No software
Muda	atividade que não agrega valor	retrabalho, espera, funcionalidade que ninguém usa
Mura	irregularidade, variação	uma semana parada e outra com cinco PRs de uma vez
Muri	sobrecarga	prazo irreal, uma pessoa por quem tudo passa

Os três se alimentam: a sobrecarga (muri) gera irregularidade (mura), que gera espera e retrabalho (muda). Atacar só o desperdício visível, sem tratar a sobrecarga e a variação, resolve pouco.

📖 Ref. Lean Enterprise Institute — Muda, Mura, Muri: https://www.lean.org/lexicon-terms/muda-mura-muri/

---

3. Os sete desperdícios e os sete princípios no software

3.1 Os sete desperdícios

Mary e Tom Poppendieck traduziram os desperdícios da fábrica para o desenvolvimento:

#	Desperdício	Exemplo num projeto da disciplina
1	Trabalho parcial	branch aberta há duas semanas, sem integrar
2	Funcionalidades extras	tela de configurações que nenhuma história pediu
3	Reaprendizado	redescobrir como o deploy funciona porque ninguém anotou
4	Transferências	tarefa que passa por três pessoas até alguém terminar
5	Atrasos	PR esperando revisão por quatro dias
6	Troca de tarefas	uma pessoa com três cartões em "Em progresso"
7	Defeitos	bug que volta de "Pronto" para "Em progresso"

Os desperdícios 1, 5 e 6 aparecem direto no quadro: cartões demais em andamento, cartões parados numa coluna de espera. É por isso que o Kanban limita o WIP (capítulo 6).

3.2 Os sete princípios

#	Princípio	Em uma frase
1	Eliminar desperdício	tudo o que não agrega valor para o cliente
2	Construir qualidade	qualidade embutida no processo, não inspecionada no fim
3	Criar conhecimento	o código e o processo ensinam a equipe; registre o que aprendeu
4	Adiar compromissos	decidir no último momento responsável, com mais informação
5	Entregar rápido	ciclos curtos, retorno cedo
6	Respeitar as pessoas	quem faz o trabalho decide como fazê-lo
7	Otimizar o todo	o sistema inteiro, não partes isoladas

Adiar compromissos não é procrastinar: é não fechar uma decisão cara (um banco de dados, uma arquitetura) antes de ter o que é preciso para decidi-la. Otimizar o todo explica por que "cada um ocupado o tempo todo" não é meta: uma equipe pode estar toda ocupada e entregar pouco, se o trabalho se acumula esperando revisão.

3.3 Duas ferramentas: 5 Porquês e PDCA

- 5 Porquês: perguntar "por quê?" repetidas vezes, a partir de um problema observado, até chegar a uma causa sobre a qual a equipe pode agir. Cinco é um número de referência, não uma regra.

  ```
  Problema: o PR #14 ficou 4 dias esperando revisão.
  Por quê? Ninguém viu que ele estava pronto.
  Por quê? A notificação do GitHub vai para um e-mail que ninguém lê.
  Por quê? Não combinamos um canal para pedir revisão.
  Por quê? O acordo de processo fala em revisão, mas não em como pedi-la.
  Ação: pedir revisão no grupo da equipe, com o link; quem pegar reage ao pedido.
  ```

- PDCA (Plan, Do, Check, Act): planejar uma mudança, executá-la, medir o efeito e ajustar. É a melhoria contínua em forma de ciclo, e é o que a retrospectiva faz a cada sprint.

📖 Ref. Lean Enterprise Institute — 5 Whys: https://www.lean.org/lexicon-terms/5-whys/

---

4. Kanban: gerir o fluxo

4.1 O que é

Kanban é um método para gerir e melhorar o fluxo de trabalho. Três ideias o definem:

- Visualizar o trabalho: um quadro que mostra os itens e em que etapa cada um está.
- Limitar o trabalho em andamento (WIP, *work in progress*).
- Gerir e melhorar o fluxo com base no que se observa.

Kanban é evolucionário: começa pelo processo que a equipe já tem e o melhora aos poucos. Não exige novos papéis nem uma reorganização.

O quadro sozinho não é Kanban. Um quadro sem limites e sem políticas é uma lista de tarefas em colunas: as pessoas continuam empurrando trabalho para frente, sem que nada obrigue a terminar.

4.2 Sistema puxado

- Sistema empurrado: quem termina uma etapa passa o item adiante, esteja a próxima etapa livre ou não. O trabalho se acumula na frente de quem está mais lento.
- Sistema puxado: cada etapa só puxa um item novo quando tem capacidade. O limite de WIP é o que materializa essa capacidade.

Na prática, puxar significa que ninguém começa um cartão novo se a coluna "Em progresso" está no limite: a pessoa ajuda a terminar o que já está lá, por exemplo revisando um PR.

4.3 Princípios e práticas

O Kanban Guide resume as práticas em três: definir e visualizar o fluxo, gerir ativamente os itens em andamento e melhorar o fluxo. As versões mais antigas do método (David Anderson) falam em seis práticas: visualizar, limitar o WIP, gerir o fluxo, tornar as políticas explícitas, implementar ciclos de retorno e melhorar colaborativamente. As duas listas descrevem a mesma coisa em granularidades diferentes.

📖 Ref. The Kanban Guide: https://kanbanguides.org/english/

📖 Ref. Agile Alliance — Kanban: https://www.agilealliance.org/glossary/kanban/

---

5. Visualizar e tornar as políticas explícitas

5.1 Colunas que refletem o trabalho real

As colunas do quadro devem ser as etapas por que o trabalho de fato passa. As da Sprint 0:

Coluna	Significado
Backlog	priorizado, fora da sprint atual
Sprint Backlog	comprometido para a sprint
Em progresso	alguém está trabalhando nele
Em revisão	PR aberto, esperando revisão
Pronto	atende à Definição de Pronto

Se o trabalho passa por uma etapa que não tem coluna (por exemplo, testar no celular de alguém antes de integrar), ele fica invisível ali. Se há uma coluna por onde nada passa, ela só confunde. Ajustem as colunas pelo que observarem.

5.2 Políticas de coluna

Uma política diz o que precisa ser verdade para um cartão entrar ou sair de uma coluna. Exemplos:

- Em progresso → Em revisão: PR aberto, CI verde, descrição do PR diz como testar.
- Em revisão → Pronto: aprovação de outro integrante, PR integrado, cartão ligado ao PR.

Escritas na descrição da coluna, as políticas encerram discussões ("isso já está pronto?") e tornam a Definição de Pronto verificável. Políticas implícitas, que só uma pessoa conhece, geram conflito e retrabalho.

5.3 Um cartão por unidade de trabalho

- Cartões pequenos, que se concluem em poucos dias, fluem melhor e dão métricas melhores. Uma história grande vira vários cartões, cada um entregando uma fatia.
- Cada cartão tem um responsável e aponta para o PR que o resolve.
- Cartão bloqueado (esperando alguém de fora, uma decisão, uma dependência) fica marcado como bloqueado, com o motivo. Bloqueio escondido é o gargalo mais difícil de ver.

---

6. Limitar o WIP e a Lei de Little

6.1 Por que limitar

Com muitos itens em andamento ao mesmo tempo:

- cada pessoa alterna entre tarefas (desperdício 6), e cada troca custa tempo de retomada;
- tudo está começado e nada está terminado (desperdício 1);
- problemas aparecem tarde, porque nada chega ao fim para ser testado e revisado;
- o tempo de entrega de cada item cresce, mesmo com todo mundo ocupado.

O limite de WIP é um teto de itens numa coluna (ou num conjunto de colunas). Quando o teto é atingido, a regra é terminar antes de começar.

6.2 A Lei de Little

A Lei de Little relaciona três médias de um sistema estável:

```
WIP médio = throughput médio × tempo médio no sistema
```

Reescrita como a aula apresentou:

```
lead time médio = WIP médio ÷ throughput médio
```

Exemplo: 12 itens em andamento e 3 concluídos por semana dão um tempo médio de 12 ÷ 3 = 4 semanas por item. Com o mesmo throughput e metade do WIP (6 itens), o tempo cai para 2 semanas. É o argumento central do limite de WIP: sem trabalhar mais rápido, reduzir o trabalho simultâneo reduz o tempo que cada item leva para sair.

Cuidados ao usar a lei:

- Ela vale para médias de longo prazo num sistema estável (o que entra, sai; itens não somem nem ficam parados para sempre). Numa sprint de três semanas, os números são aproximados.

- Serve para explicar e prever tendências, não para prometer a data de um item específico. Para isso, a distribuição dos tempos (capítulo 7) é mais útil que a média.

6.3 Qual limite usar

Não há número mágico. Um começo razoável para uma equipe de quatro pessoas:

- "Em progresso": até um item por pessoa (limite 4), ou menos, se a equipe trabalhar em par.
- "Em revisão": 2 ou 3. Se essa coluna enche, o gargalo é a revisão, e a resposta é revisar antes de começar coisa nova.

Ajustem o limite pelo que observarem. Limite nunca atingido não restringe nada; limite sempre estourado não está sendo respeitado. Os dois casos são assunto para a retrospectiva.

> Erro comum: declarar o WIP no acordo de processo e não olhar para ele. A rubrica pede WIP configurado e respeitado ao longo da sprint.

---

7. Métricas de fluxo

7.1 As quatro métricas

Métrica	Pergunta	Unidade
Lead time	quanto tempo o cliente espera, do pedido à entrega?	dias
Cycle time	quanto tempo o trabalho leva, do início à entrega?	dias
Throughput	quantos itens terminamos por período?	itens por semana
WIP	quantos itens estão em andamento agora?	itens

- Lead time começa quando o item entra no quadro (é pedido); cycle time, quando alguém começa a trabalhar nele. A diferença entre os dois é o tempo de espera antes do início.
- As definições exatas dos pontos de início e fim variam entre autores. O que importa é a equipe escolher e usar sempre as mesmas, escritas no acordo de processo.
- Contem em dias corridos, que é o que o cliente sente, ou em dias úteis, desde que sempre do mesmo jeito.

📖 Ref. Agile Alliance — Lead Time: https://www.agilealliance.org/glossary/lead-time/

7.2 Um exemplo com seis cartões

Uma sprint de 14/09 a 02/10, com os cartões concluídos:

Cartão	Entrou no quadro	Início (Em progresso)	Pronto	Lead time	Cycle time
#1	14/09	15/09	17/09	3	2
#2	14/09	15/09	22/09	8	7
#3	14/09	18/09	23/09	9	5
#4	16/09	22/09	24/09	8	2
#5	16/09	23/09	29/09	13	6
#6	21/09	24/09	30/09	9	6

Tempos em dias corridos, contando da data de início até a data de fim.

- Lead time médio: (3 + 8 + 9 + 8 + 13 + 9) ÷ 6 = 50 ÷ 6 ≈ 8,3 dias.
- Cycle time médio: (2 + 7 + 5 + 2 + 6 + 6) ÷ 6 = 28 ÷ 6 ≈ 4,7 dias.
- Tempo médio de espera antes do início: 8,3 – 4,7 ≈ 3,7 dias. Os cartões passam quase metade do lead time parados no "Sprint Backlog".
- Throughput por semana: 1 (14 a 20/09), 3 (21 a 27/09) e 2 (28/09 a 02/10), média de 2.
- WIP médio em "Em progresso", pela Lei de Little: throughput de 6 itens em 19 dias (≈ 0,32 item por dia) × cycle time de 4,7 dias ≈ 1,5 item.

O que esses números dizem: a equipe começa trabalho demais cedo (entradas no quadro em 14/09, inícios espalhados até 24/09), e a primeira semana quase não entregou (throughput 1). É uma boa pergunta para a retrospectiva: o que travou a primeira semana?

7.3 Média, distribuição e percentil

Médias escondem os casos que mais incomodam. O #2 levou 7 dias de ciclo, mais que o dobro da maioria. Duas formas de olhar além da média:

- Gráfico de dispersão: cada cartão é um ponto (data de conclusão × cycle time). Pontos muito acima dos outros são os casos a investigar.
- Percentil: "85% dos cartões ficaram prontos em até N dias". Com poucos cartões, como nesta sprint, basta ordenar os tempos e olhar os maiores.

7.4 Idade do item em andamento

Lead e cycle time só existem depois que o cartão termina. Durante a sprint, a métrica útil é a idade: há quantos dias um cartão está em "Em progresso" ou "Em revisão". Um cartão mais velho que o cycle time costumeiro da equipe é um sinal para agir agora, e não só na retrospectiva.

7.5 Diagrama de fluxo cumulativo

O diagrama de fluxo cumulativo (CFD) mostra, dia a dia, quantos itens estão em cada coluna, em faixas empilhadas. Como ler:

- a distância vertical entre as faixas de "Em progresso" e "Pronto" é o WIP daquele dia;
- a distância horizontal entre a entrada e a saída de uma faixa é, aproximadamente, o tempo que os itens ficam naquela etapa;
- uma faixa que engorda é um gargalo: entra mais do que sai daquela coluna;
- a inclinação da faixa "Pronto" é o throughput.

Para montar um CFD à mão: uma vez por dia, contem os cartões de cada coluna e anotem numa planilha; o gráfico de área empilhada da planilha é o CFD.

7.6 Quando o quadro não registra: medir pelo repositório

Um quadro que não se move não produz métrica nenhuma: sem item em "Pronto", não há throughput nem cycle time. Isso não deixa a equipe sem dados — o repositório registra fluxo o tempo todo, e o `gh` (a linha de comando do GitHub) o devolve em JSON:

```
git log --format='%ad %h %s' --date=short              # ritmo de integração
gh pr list --state all --json createdAt,mergedAt,reviews  # espera por revisão
gh issue list --state all --json createdAt,closedAt       # idade dos itens
gh run list --json conclusion,createdAt,updatedAt         # CI: falhas e tempo de retorno
```

O tempo entre a abertura e a integração de um PR é um cycle time observado; a diferença entre a data de abertura da issue e a do PR que a fecha é um lead time observado. É menos completo que o quadro — não enxerga a espera antes de alguém começar —, e é por isso que o quadro existe. Mas serve para a primeira retrospectiva e, principalmente, para mostrar a distância entre o processo declarado e o praticado.

O projeto de referência tem essa coleta feita, com o comando ao lado de cada número: `processo/metricas-01.md`, em `github.com/fmarquesfilho/musi`.

---

8. Gestão visual e gargalos

8.1 O quadro como radiador de informação

Gestão visual é deixar o estado do trabalho visível para todos, sem precisar perguntar. Um quadro bem mantido responde, num olhar:

- o que cada pessoa está fazendo;
- o que está parado e por quê;
- quanto falta para o compromisso da sprint.

Isso só funciona se o quadro estiver atualizado. Cartões movidos só no último dia não mostram nada durante a sprint, e a rubrica considera exatamente isso: cartões movidos ao longo da sprint.

8.2 Sinais de gargalo

Sinal no quadro	O que costuma significar
"Em revisão" sempre cheia	revisão é o gargalo; revisar vira prioridade
Cartão com a mesma pessoa há muitos dias	cartão grande demais, ou bloqueado sem aviso
"Em progresso" acima do limite	limite não respeitado, ou limite irreal
Muitos cartões voltando de "Pronto"	Definição de Pronto frouxa, defeitos
"Sprint Backlog" que não diminui	compromisso maior que a capacidade

O gargalo define a vazão do sistema inteiro. Acelerar uma etapa que não é o gargalo só aumenta a fila na frente dele (princípio 7, otimizar o todo).

---

9. Retrospectivas

9.1 Para que servem

A retrospectiva é o momento em que a equipe inspeciona como trabalhou e decide o que mudar. No Scrum, fecha cada sprint; no Kanban, é um dos ciclos de retorno. É o PDCA aplicado ao processo: o que planejamos, o que aconteceu, por quê, e o que muda.

📖 Ref. Scrum Guide — Sprint Retrospective: https://scrumguides.org/scrum-guide.html

9.2 Uma estrutura em cinco etapas

Esther Derby e Diana Larsen, em *Agile Retrospectives*, propõem uma sequência que funciona bem para uma hora de conversa:

1. Preparar o terreno: lembrar o objetivo e o combinado (todos falam, o foco é o processo, não as pessoas).
2. Coletar dados: fatos da sprint, com o quadro e as métricas abertos.
3. Gerar entendimento: discutir as causas dos principais problemas (5 Porquês).
4. Decidir o que fazer: escolher poucas ações.
5. Encerrar: registrar e combinar como as ações serão acompanhadas.

9.3 Formatos para coletar dados

Formato	Perguntas
Começar, parar, continuar	O que começar a fazer? Parar? Continuar?
4Ls	O que gostamos, aprendemos, sentimos falta e desejamos?
Veleiro	O que nos impulsionou (vento), o que nos segurou (âncora), que riscos vemos (rochas)?

O formato importa menos que o que vem depois. Trocar de formato de vez em quando evita respostas automáticas. O Retromat tem dezenas de atividades para cada etapa.

📖 Ref. Retromat: https://retromat.org/en/

9.4 Fatos, causas e ações

A rubrica da Sprint 1 pede `docs/retrospectiva-01.md` com fatos observados, causas discutidas e ações com responsável e prazo. Um exemplo de trecho:

```
## Fatos
- Cycle time médio de 4,7 dias; o cartão #2 levou 7.
- "Em revisão" passou do limite (3) em 4 dos 15 dias úteis.
- Na primeira semana, só 1 cartão foi concluído.

## Causas
- O #2 misturava tela e integração com a API; era grande demais (5 Porquês na reunião).
- Pedidos de revisão ficavam só na notificação do GitHub, que ninguém acompanha.

## Ações
| Ação | Responsável | Prazo |
|---|---|---|
| Quebrar histórias com mais de 3 dias estimados antes de puxá-las | Ana | início da Sprint 2 |
| Pedir revisão no grupo da equipe, com o link do PR | todos; Bruno acompanha | a partir de 05/10 |
```

Boas ações são poucas (até três), concretas e verificáveis na próxima retrospectiva. "Melhorar a comunicação" não é ação; "pedir revisão no grupo, com o link do PR" é.

> Erro comum: retrospectiva que registra impressões ("foi corrido", "a comunicação pode melhorar") sem fato, causa ou ação. A rubrica classifica isso como suficiente, no máximo.

> Erro comum: ações sem responsável. Uma ação que é de todos acaba não sendo de ninguém.

> Erro comum: não revisar as ações da retrospectiva anterior. A próxima retrospectiva começa pelo que foi combinado na última: foi feito? Funcionou?

📖 Ref. Esther Derby e Diana Larsen, *Agile Retrospectives: Making Good Teams Great* (Pragmatic Bookshelf, 2006).

9.5 Um exemplo real, com números

`processo/retrospectiva-01.md`, no projeto de referência (`github.com/fmarquesfilho/musi`), é uma retrospectiva escrita sobre dados do próprio repositório, e não sobre impressões. Os fatos que ela abre:

Fato	Número
Cartões no quadro, todos em "Todo"	3, sem nenhuma mudança de status em 22 dias
Commits direto na `main`, sem pull request	32 de 43 (74%)
Único pull request	integrado em 48 min, sem revisão registrada
`main` vermelha depois de uma falha de CI	25,8 h
Issues de débito abertas e paradas	3, há 22 dias

O interessante é o que esses números contradizem: o acordo de processo prometia limites de WIP em colunas que o quadro não tinha, revisão 24 h depois em equipe de um, e métricas tiradas dos *Insights* do GitHub Projects — que, com o quadro parado, não produzem nada.

A causa raiz, pelos 5 Porquês, não foi falta de disciplina: o quadro estava **ao lado** do trabalho, e não no caminho dele. Enquanto commitar direto na `main` for o caminho mais curto, nenhum quadro fica atualizado. Daí a ação principal ser mecânica (proteger a branch, exigindo pull request), e não uma promessa de comportamento.

Vale ler junto com o `processo/acordo-de-processo.md` do mesmo projeto: a retrospectiva altera três pontos dele, e é esse ciclo — acordo, realidade, acordo revisado — que a rubrica procura.

---

10. No projeto: o GitHub Projects

10.1 Configurar o quadro

- Layout de quadro (*board*) agrupado pelo campo Status, com as colunas da seção 5.1.
- Limite de itens por coluna: no menu da coluna, é possível definir um limite. O GitHub mostra a contagem e o limite no topo da coluna e destaca quando ele é ultrapassado, mas não impede que alguém adicione cartões além dele. Respeitar o limite continua sendo um combinado da equipe.
- Políticas de coluna: na descrição de cada coluna.

📖 Ref. GitHub Docs — Customizing the board layout: https://docs.github.com/en/issues/planning-and-tracking-with-projects/customizing-views-in-your-project/customizing-the-board-layout

10.2 Registrar as datas para as métricas

O GitHub Projects não calcula lead time nem cycle time. Duas formas de ter os dados:

- Campos de data no projeto, por exemplo "Início" e "Fim", preenchidos ao mover o cartão para "Em progresso" e para "Pronto". Exportados para uma planilha, dão os cálculos do capítulo 7.
- As datas dos PRs: abertura (aproximação do início) e integração (fim). O `gh` lista as duas com `gh pr list --state merged --json number,createdAt,mergedAt`.

A data de entrada no quadro (início do lead time) é a data em que o item foi adicionado ao projeto.

📖 Ref. GitHub Docs — About date fields: https://docs.github.com/en/issues/planning-and-tracking-with-projects/understanding-fields/about-date-fields

10.3 Insights

A aba Insights do projeto tem gráficos atuais (por exemplo, itens por responsável) e históricos. O gráfico histórico padrão é um *burn up*, com itens abertos, concluídos e não planejados ao longo do tempo. Ele não separa por coluna do quadro; para um CFD por coluna, use a contagem diária numa planilha (seção 7.5).

📖 Ref. GitHub Docs — About insights for Projects: https://docs.github.com/en/issues/planning-and-tracking-with-projects/viewing-insights-from-your-project/about-insights-for-projects

10.4 Evidência de XP no repositório

A rubrica pede ao menos uma prática de XP com evidência:

Prática	Evidência verificável
Programação em par	commits com `Co-authored-by:`

Prática	Evidência verificável
TDD	commit do teste antes do commit da implementação
Refatoração	PRs de refatoração, com os testes passando antes e depois
Integração contínua	CI em todo PR, branches curtas integradas com frequência

📖 Ref. Martin Fowler — Pair Programming: https://martinfowler.com/bliki/PairProgramming.html

📖 Ref. Martin Fowler — Continuous Integration: https://martinfowler.com/articles/continuousIntegration.html

---

11. Exercícios e dúvidas frequentes

11.1 Perguntas de fixação

1. Qual a diferença entre muda, mura e muri? Dê um exemplo de cada no projeto.
2. Por que limitar o WIP reduz o lead time, sem ninguém trabalhar mais rápido?
3. Qual a diferença entre lead time e cycle time? O que a diferença entre eles mede?
4. Por que a Lei de Little não serve para prometer a data de um item específico?
5. Como um gargalo aparece num diagrama de fluxo cumulativo?
6. O que distingue uma ação de retrospectiva de uma impressão?

11.2 Exercícios

1. Com os números da seção 7.2, calcule o lead time médio se o #5 tivesse entrado no quadro em 22/09. O que muda na conclusão sobre a espera?
2. Uma equipe tem, em média, 8 itens em andamento e conclui 4 por semana. Qual o tempo médio de ciclo? E se o WIP cair para 4?
3. Escrevam as políticas de entrada e saída de cada coluna do quadro do projeto.
4. Apliquem os 5 Porquês ao cartão que mais demorou na sprint.

11.3 Dúvidas frequentes

- Kanban substitui o Scrum no projeto? Não. O ritmo de sprints continua; o Kanban organiza o fluxo dentro delas.
- O GitHub impede passar do limite de WIP? Não, só mostra. O limite é respeitado por acordo.
- Precisamos de um CFD na entrega? A rubrica pede WIP respeitado, cartões movidos ao longo da sprint e gargalos visíveis. Métricas e CFD são a melhor forma de mostrar isso na retrospectiva e no vídeo.
- Dias corridos ou úteis? Qualquer um, desde que sempre o mesmo, e escrito no acordo.

Referências

- The Kanban Guide: https://kanbanguides.org/english/
- Agile Alliance — Kanban: https://www.agilealliance.org/glossary/kanban/ · Lead Time: https://www.agilealliance.org/glossary/lead-time/ · Extreme Programming: https://www.agilealliance.org/glossary/xp/
- Lean Enterprise Institute — 5 Whys: https://www.lean.org/lexicon-terms/5-whys/ · Muda, Mura, Muri: https://www.lean.org/lexicon-terms/muda-mura-muri/
- Scrum Guide: https://scrumguides.org/scrum-guide.html
- Retromat: https://retromat.org/en/
- Martin Fowler — Pair Programming: https://martinfowler.com/bliki/PairProgramming.html · Continuous Integration: https://martinfowler.com/articles/continuousIntegration.html
- GitHub Docs — board layout: https://docs.github.com/en/issues/planning-and-tracking-with-projects/customizing-views-in-your-project/customizing-the-board-layout · date fields: https://docs.github.com/en/issues/planning-and-tracking-with-projects/understanding-fields/about-date-fields · Insights: https://docs.github.com/en/issues/planning-and-tracking-with-projects/viewing-insights-from-your-project/about-insights-for-projects

Livros

- Mary e Tom Poppendieck — *Implementing Lean Software Development*
- David J. Anderson — *Kanban: Successful Evolutionary Change for Your Technology Business*
- Jeffrey Liker — *The Toyota Way*
- Donald Reinertsen — *The Principles of Product Development Flow*
- Esther Derby e Diana Larsen — *Agile Retrospectives: Making Good Teams Great*
- Daniel Vacanti — *Actionable Agile Metrics for Predictability*